
Ribbon Filter Construction at Bloom-Filter Speed: Partition-Instead-of-Sort, and Where Layout Tricks Cannot Work

Brian Sam-Bodden
Integrallis Software

Abstract

Probabilistic membership filters — Bloom filters and their modern successors — answer a single question: is this key possibly in this set? Storage systems keep one in front of nearly every file on disk so that a lookup that would find nothing can skip the disk read entirely. Newer filter designs, such as ribbon and binary fuse filters, deliver the same accuracy as Bloom filters in far less memory, within a few percent of the theoretical minimum. Yet they have not displaced Bloom filters in practice, and the reason is construction speed: these filters are immutable, storage engines rebuild them constantly as data is compacted, and building a ribbon filter costs roughly four times the CPU of building a Bloom filter.

This paper asks where that construction time actually goes on modern processors, and where it can and cannot be won back. Two negative results come first: batching and cache-layout tricks do not speed up filter queries or Bloom filter construction, because the memory stalls in both are independent of one another and out-of-order hardware already overlaps them. Ribbon construction is different. Its stalls form dependent chains — each step of its linear-system elimination waits on the one before — and dependent chains are the one memory cost that hardware cannot hide but reordering can remove. A single counting pass that groups keys by the region of the filter they touch, a weaker and roughly four-times-cheaper form of the full sort used in prior work, makes sequential ribbon construction $2.05\text{--}2.24\times$ faster at 100–400 million keys. We further prove that the finished filter is bit-for-bit identical regardless of insertion order, which lets a parallel builder verify its own correctness with a checksum: with 16 threads, a ribbon filter now builds as fast as a single thread inserts into a Bloom filter. Every number in the paper re-derives mechanically from committed raw measurements.

1 Introduction

A membership filter is a small in-memory summary of a large set of keys. Ask it whether a key is present and it answers either “definitely not” or “probably yes,” and the first answer is the valuable one: a database that keeps a filter in front of each data file can skip reading any file whose filter says no. Bloom filters [Bloom 1970] have played this role for five decades, but they spend about 44% more memory than the theoretical minimum for a given error rate. A newer generation of filters — xor, binary fuse, and ribbon filters — closes that gap to within 10% and, in ribbon’s bumped variants, under 1% [Dillinger & Walzer 2021; SEA 2022].

There is a catch, and it is the subject of this paper. The new filters are static: they are built once over a fixed key set and never updated. That fits how write-optimized databases actually work — data files are immutable, and background compaction continuously rewrites them — but it means every rewrite also rebuilds the file’s filter, so construction speed matters as much as query speed. And construction is where the new filters lose: building a ribbon filter costs roughly four times the CPU of

building a Bloom filter (RocksDB measures 140 vs. 32 ns per key [RocksDB blog 2021]), which is why production systems still default to Bloom filters and reserve the space-efficient kind for data they rarely rewrite. The people closest to these structures say as much: the parallel-BuRR authors note that “a large part of the construction time is spent on parallel sorting” [arXiv:2411.12365], and the most recent proposals concede they build several-fold slower than the best existing builders [ZOR, arXiv:2602.03525].

This paper asks which parts of that cost are real — pinned to the hardware — and which parts can be won back. We answer with three experiments whose protocols and success thresholds were fixed before any data was collected, and the answer reduces to one mechanical rule: optimization pays exactly where memory stalls form data-dependent chains, because independent stalls are already overlapped by out-of-order hardware and software prefetch, while dependent chains defeat both.

Contributions.

1. We could not make batched probing win, and the reason turned out to matter more than the attempt (§3). Miss-dominated point probes run into the per-core limit on outstanding misses; software prefetch was already at $\sim 88\%$ of that ceiling when we measured it. Cross-key SIMD batching loses to plain per-key probing even with every address L1-resident. Bulk Bloom construction fell the same way, for the same underlying reason: its misses are independent, so the memory system hides them. Radix-partitioned construction in the style of Schmidt et al. (PVLDB 2021) topped out at $1.45\text{--}1.71\times$ over per-key insertion, a prefetch loop that RocksDB ships (but never evaluated) recovered most of that with zero transient memory, and our read-free register-aggregated apply — the idea we had the most hope for — came in $1.5\times$ slower than partitioned RMW, because partitioning had already turned the target misses into L2 hits.
2. Ribbon construction is where the same playbook finally pays (§4). Homogeneous ribbon banding at scale burns 5.8 DRAM misses per key in dependent row-reduction chains, and those chains are exactly what prefetch cannot hide. A single counting pass that groups keys into L2-sized start-windows — approximate order, nothing more — recovers 98.3–98.4% of the miss reduction that the full start-position sort (SGAUSS [ESA 2019], BuRR) buys, at 5.5–6.1 ns/key where BuRR’s own ips2ra sorter costs 23.9–28.1. End to end that makes construction $2.05\text{--}2.24\times$ faster than the reference prefetch-pipelined build. Window size barely matters; totals stay within 13% across 2^{13} through 2^{20} slots.
3. Banding turns out to be order-independent, and that observation carries real weight (§5). With first-coefficient-always-one pivoting, the solved filter is bit-identical under any insertion order — we prove this for linearly independent rows and saw it hold in 54 of 54 runs. So reordering is output-neutral by construction, and a slot-range parallel variant that defers boundary keys (at most 0.23% of them) can verify itself with a checksum: $4.25\times$ faster banding at 8 threads, $7.56\times$ at 16, against its own single-thread run ($3.5\times$ and $6.3\times$ against the best sequential banding). The total lands at 11.9 ns/key at 100M keys.
4. Along the way we ended up producing the first actual evaluation of the construction prefetch RocksDB ships behind a “TODO: verify/validate” comment (§6). It is worth $1.5\text{--}1.7\times$ on unsorted banding. Partitioning mostly subsumes it.

Everything ran on one i9-14900HX (AVX2, no AVX-512), and we deliberately measured through pinned reference harnesses rather than our own benchmark code wherever possible — fastfilter_cpp, whose ribbon implementations come from the ribbon author himself, plus BuRR and FastLanes. Raw artifacts, a claims ledger, and automated integrity checks (information-theoretic FPR bounds, provenance) live next to the code, because an earlier version of one of our own instruments fooled us and we only caught it by cross-checking against a second harness. ARM replication is scripted and waiting on cloud capacity.

2 Background

Ribbon banding. A ribbon filter solves a banded GF(2) system. Each key hashes to a start column s and a w -bit coefficient row, and insertion XOR-reduces that row against whatever is already stored until an empty slot absorbs it. The lineage matters here. Dietzfelbinger and Walzer showed that sorting rows by start position makes banded elimination linear (SGAUSS). The ribbon paper’s whole

pitch was that you don’t need the sort — insert on the fly, in arbitrary order. BuRR then brought the sort back (ips2ra, by bucket) because bumping needs it. Homogeneous ribbon, the variant we target, never fails construction and stores no per-key result rows.

Binary fuse construction partially sorts keys by segment explicitly to reduce cache misses [Graf & Lemire, JEA 2022]; our Phase-0 characterization measures that fix directly (plain xor: 9.45 misses/key; binary fuse: 1.08).

3 Where layout tricks cannot work

3.1 Point probes are MLP-bound

Experiment E0 compared probe strategies over identical split-block Bloom filters (Parquet geometry), 32KiB up to 512MiB, with the protocol frozen before any timing run. Our vertical, lane-structured batch probe does auto-vectorize from plain Rust — the packed-SIMD disassembly is committed. It still loses. At every size, even with every address forced L1-hot, per-key scalar probing wins (665 vs. 555 Mops/s), because a split-block check is already SIMD within a single key, and an AVX2 gather cannot beat eight independent scalar loads that the out-of-order window overlaps for free. The RAM-scale picture is starker: the identical instruction stream runs 5–6× faster when L1-hot. Probing is bound by memory-level parallelism, full stop. Prefetch was already at roughly 88% of the single-core outstanding-miss ceiling. We measured it. There is no compute bottleneck left for a layout to remove.

3.2 Blocked-Bloom construction is prefetch-hidable

Experiment E1a measured five bulk builders at 1M to 429M keys, gated on bit-identical output (OR is commutative, so any deviation is a bug). Per-key insertion does degrade at scale — one random read-modify-write miss per key, 11.5 ns/key at 100M, IPC 0.48 — so there was a real cost to attack. The attacks mostly failed. Radix-partitioned construction after Schmidt et al. reached only 1.45–1.71×. A software-pipelined prefetch insert, the shape RocksDB ships, hit 6.1–8.9 ns/key with zero transient memory and actually beat partitioning up to 100M keys. And the variant we designed this experiment around — a read-free apply that partitions pre-merged (block, mask) pairs, OR-aggregates each block in registers, and issues exactly one store per block without ever reading filter memory — came in 1.5× slower than partitioned RMW. Non-temporal stores changed nothing. In hindsight the accounting is obvious: partitioning had already turned the expensive misses into L2 hits, so the extra grouping pass was spending real time to save cheap loads. Bloom construction misses are independent, and the memory system hides independent misses better than we could.

4 Partition-instead-of-sort for ribbon

4.1 The target, measured

Phase-0 characterization (fastfilter_cpp binary, 3 reps) shows that homogeneous ribbon construction at 100M keys costs 50.6 ns/key at IPC 0.64 with 5.81 cache-misses/key — banding walks a random start per key through a 1.3GB working set, and each row-reduction step depends on the previous XOR. Thus prefetch cannot dissolve the chain as Bloom’s independent probes can.

4.2 Design

We only permute the input. One counting pass buckets keys by $\text{floor}(s / 2^{16})$ — windows of 2^{16} slots, roughly 768KiB of banding state, under half an L2 — recomputing the hash on the fly instead of materializing pairs; a second pass writes the keys back out in window order. The untouched reference kernel, prefetch pipeline and all, then consumes the permuted array. Everything downstream — banding, back-substitution, queries — stays the ribbon author’s code.

4.3 Results

Table 1 (100M keys; 400M artifacts). The partition pass costs 5.5-6.1 ns/key against 23.9-28.1 for ips2ra in our driver (17.3 inside BuRR’s own bench); banding lands within 13% of banding-after-full-

Table 1: Homogeneous ribbon construction, 100M keys, ns/key (mean of 3; raw artifacts in repository). Banding perf counters scoped to the banding phase.

strategy	reorder	banding	total	miss/key
unsorted, no prefetch	—	76.9	80.8	5.81
unsorted + prefetch (ref.)	—	46.1	49.6	5.75
full sort (std::sort)	76.8	15.5	96.1	0.074
full sort (LSD radix)	23.4	14.7	41.7	0.076
full sort (ips2ra)	23.9	13.4	40.7	0.075
partitioned (ours)	5.5	15.2	24.2	0.170
parallel, 8T (ours)	5.5	4.3	14.6	—
parallel, 16T (ours)	5.7	2.4	11.9	—

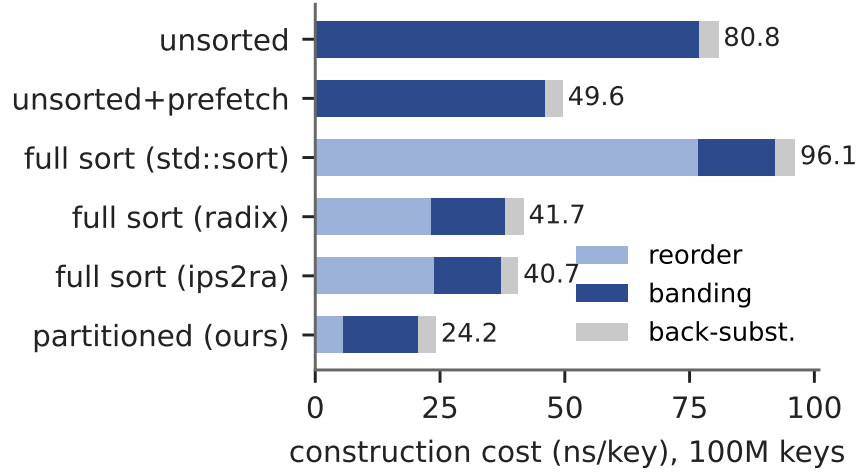


Figure 1: Construction cost by phase, 100M keys. The partition pass replaces the sort at $4\times$ lower reorder cost while banding stays within 13% of banding-after-full-sort.

sort (15.2 vs. 13.4 ns/key), i.e., the partition recovers 98.3-98.4% of the sort miss reduction (5.75 $\rightarrow$ 0.17 vs. 0.075 misses/key). End-to-end we get 24.2 vs. 49.6 (reference) and 40.7 (best full sort): $1.68\times$ faster than the best full sort at both sizes. As an anchor, BuRR builds end-to-end at about 70 ns/key at its top recursion level on this machine (roughly 75.5 across all levels). It is a different structure — $r=8$, about 1% space overhead, bumping — whereas our homogeneous configuration spends 7.63 bits/key at 0.81% FPR, about 9.5% overhead. A window sweep from 2^{13} to 2^{20} slots moves totals only between 23.8 and 26.9 ns/key — no tuning cliff — and our registered choice of 2^{16} turns out to have been conservative; 2^{14} does mildly better. (Table 1, Figure 1.)

5 Order-independence and parallel construction

Proposition 1 (Order-independence) *Let $E = \{(r_k, b_k)\}$ be equations over $\text{GF}(2)^m$ whose rows are linearly independent, each r_k carrying a leading 1 at its start position. Then banding (store each reduced row at the slot of its leading position) followed by back-substitution with all free slots fixed to zero yields the same solution Z under every insertion order of E .*

Proof sketch. The occupied-slot set equals the canonical pivot set of $V = \text{span}\{r_k\}$, which depends only on V ; the stored system is row-equivalent to E ; and pinning the order-independent free coordinates to zero extends E to a full-rank system with a unique solution. (Full argument in the repository, docs/order-independence-proof.md; with dependent rows the enforced subset may differ by order, which the per-run fingerprint gate detects.)

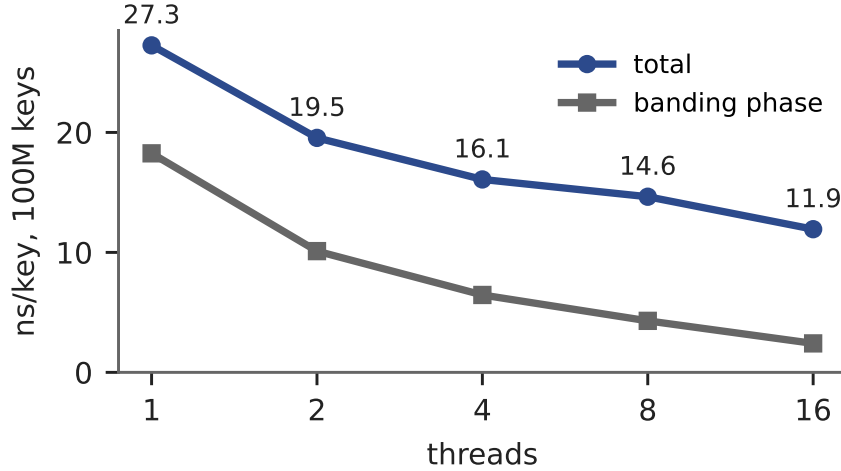


Figure 2: Slot-range parallel construction, 100M keys; every point bit-identical to the sequential build.

Empirically: for all sequential and parallel runs (sizes up to 400M, six orderings, thread counts to 16), the banded solution is exactly byte-identical under an FNV fingerprint. Reordering is therefore provably output-neutral (the linear algebra version of OR-commutativity) and it converts parallel construction verification from a distributed-systems problem to a checksum. Our parallel version assigns disjoint window ranges (split at the same key count) to threads; keys starting in 2^{14} slots of a range boundary switch to a sequential tail (0.015 - 0.226% of keys), following parallel BuRR’s boundary idea. Banding scales $1.81/2.83/4.25/7.56\times$ at 2/4/8/16 threads against its own T=1 (3.5/6.3 $\times$ at 8/16 threads against the best sequential banding; the parallel path carries $\sim 20\%$ defer/copy overhead); the residual is Amdahl’s: the sequential counting pass (5.7) and back-substitution (3.8 ns/key at 16 threads) now dominate the 11.9 ns/key total. Both are parallelizable (counting sort trivially; back-substitution as in parallel BuRR) — future work.

6 Construction prefetch, evaluated

RocksDB ships a software-pipelined construction prefetch behind a comment reading “TODO: verify/validate” and the reference harness enables the same pipeline above 1500 slots. Our no-prefetch control isolates it: 1.5-1.7 $\times$ on unsorted banding (76.9 $\rightarrow$ 46.1 ns/key at 100M). It attacks the start-locating miss but can’t touch the dependent reduction chain, which is why partitioning dominates here while the similar prefetch strategy wins for Bloom construction (§3).

7 Related work

Sorted banded elimination: Dietzfelbinger & Walzer (ESA 2019), BuRR (SEA 2022), parallel BuRR (arXiv:2411.12365). Arbitrary-order banding and production engineering: ribbon (arXiv:2103.02515), RocksDB. Cache-conscious filter design: buffered Bloom filters (Canim et al., ADMS 2010), radix-partitioned filters (Schmidt et al., PVLDB 2021), binary-fuse segment sort (JEA 2022), xor-filter buffered scatter (JEA 2020); the generic two-phase partition-then-accumulate pattern appears as propagation blocking (Beamer et al., IPDPS 2017) with write-combining from Wassenberg & Sanders (Euro-Par 2011). SIMD probe-side layout: Polychroniou & Ross (DaMoN 2014), Lang et al. (PVLDB 2019), split-block Bloom (arXiv:2101.01719). Our delta to BuRR is precise: BuRR sorts fully (ips2ra) to enable threshold-based bumping and tighter space; we show that for homogeneous ribbon — the variant RocksDB ships — a counting-pass partition captures 98% of the sort’s locality at 4 $\times$ less reorder cost, with output neutrality making the transformation and its parallelization verifiable.

8 Limitations and threats to validity

Single x86 machine by now (AVX2 Raptor Lake); ARM replication is scripted and awaiting cloud capacity, and all claims are scoped accordingly. We accelerate the *homogeneous* ribbon ($\sim 9.5\%$ space overhead at $r=7$) in comparison to BuRR (which is sub-1% overhead); the comparison with BuRR is an anchor, not an apples-to-apples comparison. Sort baselines use BuRR’s own sequential `ips2ra`, but not its parallel form. Our parallel comparison is limited to banding, and its headline ratios are against the parallel path’s own single-thread run. The order-independence proposition is proved for linearly independent rows, and the dependent case is handled empirically by the fingerprint gate. LSM end-to-end integration (build-during-compaction) is future work.

9 Conclusion

On the modern OoO x86, memory-layout optimization for filters pays in exactly one place that we could find — and it is the place that matters: the dependent-miss chains of ribbon construction, the very cost that keeps space-optimal filters out of production write paths. A one-pass partition closes 98% of the gap the literature closes with a full sort, at a quarter of the price; the order-independence makes it safe and its parallelization checkable; and 16 threads build ribbon filters as fast as one thread inserts into a Bloom filter. The two nulls are not failures of the method but the map that located the win.

Artifacts. All protocols, amendments, raw measurements, and integrity tooling are in the accompanying repository; `reproduce.sh` re-derives every number in this paper from committed raw artifacts.

References

- [1] Peter C. Dillinger and Stefan Walzer. Ribbon filter: Practically smaller than bloom and xor. *arXiv:2103.02515*, 2021.
- [2] Peter C. Dillinger, Lorenz Hübschle-Schneider, Peter Sanders, and Stefan Walzer. Fast succinct retrieval and approximate membership using ribbon. In *SEA*, 2022.
- [3] Peter C. Dillinger. Ribbon filter. RocksDB blog, rocksdb.org/blog/2021/12/29/ribbon-filter.html, 2021.
- [4] Christopher Becht, Hans-Peter Lehmann, and Peter Sanders. Brief announcement: Parallel construction of bumped ribbon retrieval. *arXiv:2411.12365*, 2024.
- [5] Antoine Limasset. ZOR filters. *arXiv:2602.03525*, 2026.
- [6] Tobias Schmidt, Maximilian Bandle, and Jana Giceva. A four-dimensional analysis of partitioned approximate filters. In *PVLDB 14(11)*, 2021.
- [7] Martin Dietzfelbinger and Stefan Walzer. Efficient Gauss elimination for near-quadratic matrices with one short random block per row, with applications. In *ESA*, 2019.
- [8] Thomas Mueller Graf and Daniel Lemire. Binary fuse filters: Fast and smaller than xor filters. *ACM JEA*, 27, 2022.
- [9] Mustafa Canim, George A. Mihaila, Bishwaranjan Bhattacharjee, Christian A. Lang, and Kenneth A. Ross. Buffered Bloom filters on solid state storage. In *ADMS@VLDB*, 2010.
- [10] Thomas Mueller Graf and Daniel Lemire. Xor filters: Faster and smaller than bloom and cuckoo filters. *ACM JEA*, 25, 2020.
- [11] Scott Beamer, Krste Asanović, and David Patterson. Reducing PageRank communication via propagation blocking. In *IPDPS*, 2017.
- [12] Jan Wassenberg and Peter Sanders. Engineering a multi-core radix sort. In *Euro-Par*, 2011.

- [13] Orestis Polychroniou and Kenneth A. Ross. Vectorized bloom filters for advanced simd processors. In *DaMoN*, 2014.
- [14] Harald Lang, Thomas Neumann, Alfons Kemper, and Peter Boncz. Performance-optimal filtering: Bloom overtakes cuckoo at high throughput. *PVLDB*, 12(5), 2019.
- [15] Jim Apple. Split block bloom filters. arXiv:2101.01719; Apache Parquet format specification, 2021.